

AI Coding 关键跃迁

2024-2026：从代理工作流到代理工程

在 AI 编程的无数概念中,真正**重塑了程序员身份**: 觉醒 → 狂热 → 约束 → 编排。

四次跃迁总览

代理 workflow

Agentic Workflow

2024

编码者 → 管理者

氛围编码

Vibe Coding

2025.2

管理者 → 验收官

规范驱动开发

Spec-Driven Dev

2025 H2

验收官 → 架构师

代理工程

Agentic Engineering

2026.2

架构师 → 编排者

我们正处于第三次跃迁向第四次跃迁过渡的阶段。

- ❏ **贯穿四次跃迁的一条暗线：**模型能力每隔数月跃进一次,每一套"最佳实践"都自带保质期。没有放之四海而皆准的答案——只有结合你的团队认知和项目本质的持续适应。

跃迁 1 — 代理工作流 Agentic workflow (2024)

AI 从工具变为数字自主体

吴恩达(Andrew Ng)的关键论断:

"未来 AI 的进步将更多来自工作流的优化,而非参数的增长。"

GPT-3.5 配合代理工作流,可以超越 GPT-4 的零样本直接输出——工程架构比模型本身更重要。

反思 Reflection

工具使用 Tool Use

规划 Planning

多代理协作 Multi-Agent

四大代理模式成为行业标准词汇。

跃迁 1 — 为什么这是一切的起点

从"被调用"到"被委托"

吴恩达之前

AI 是一问一答的聪明补全,无状态,无记忆。

吴恩达之后

AI 是能自主思考、使用工具、检查错误的 Agent。

没有"代理"这个认知转变,后续一切都不会发生——你没法对工具"放手"(就没有 Vibe Coding),不需要给工具"定规矩"(就没有 SDD),更不需要给工具"造生产线"(就没有代理工程)。

吴恩达的四种模式在当时是金科玉律——但仅仅一年后,模型自身已经学会了更复杂的规划和推理,框架正在被超越。**每一次跃迁的"答案"都有保质期。**

跃迁 2 — Vibe Coding (2025.2)

一条推文引爆的方法论内战

"我称之为 `vibe coding`,完全沉浸在氛围中.....忘记代码的存在。"

— Andrej Karpathy(OpenAI 联合创始成员、前 Tesla AI 总监)

这不是一个普通人说的话。当 AI 领域的图腾级人物说"忘记代码",整个行业被迫选边站。

拥抱派的数据很诱人：44% 开发者手写代码不足 10%,Y Combinator 1/4 创业公司 95%+ 代码由 AI 生成。

然后,灾难来了。

跃迁 2 — 真实的灾难与冰冷数据

审查的负担超过了生成的速度



核心减速

资深开发者反而慢了 **19%**——尽管自认为更快了



代码崩塌

AI 共同编写代码的重大缺陷率激增 **1.7 倍**



安全风险

AI 生成代码约 **45%** 含安全漏洞



知识枯竭

Stack Overflow 月提问量暴跌 **90%**, 文档流量 -40%

代码产出暴增,但人类的审查带宽没有增长。更多代码被"接受"而非"理解"。

灾难的根因不是程序员不努力——是模型能力跑在了组织适应力前面。每个团队消化新能力的速度不同,这里没有统一节奏。

跃迁 2 — 文化冲击波与选边站

争论焦点已从"用不用"变成"怎么用"



拥抱派

Karpathy, Replit

"接受指数级变化,忘记代码的存在"



纪律派

Kent Beck, Simon Willison

"AI 是精灵,需要瓶子——测试与规范"



务实派

大多数一线开发者

60% 使用 AI,但仅 0-20% 完全委托

Kent Beck 发现了一个荒诞现象：AI 会删除测试来让测试通过。

到 2025 年底,连 Karpathy 自己都修正立场。没有人能退回"不用 AI"的时代——但也没有人能给出一个通用的"正确用法"。答案因团队而异。

跃迁 2 — 认知债务(Cognitive Debt)

比技术债务更危险：你不知道自己不知道什么

当你"Accept All"让 AI 写完代码而不去理解它,你积累的不是技术债务——而是对自己系统的无知。

❏ 隐性契约被打破：**程序员必须理解自己系统的运行逻辑**。当这个契约被打破,任何后续变更都变成在黑暗中摸索。

15 年积累的公开知识体系——多元解法、权衡分析、边界案例、社区审查——正在消散为私有的 AI 对话。

AI Coding本质上都在做同一件事：**在不手写代码的前提下,重建人类对系统的理解力和控制力。**

跃迁 3 — 规范驱动开发 SDD (2025 H2)

代码不再是核心资产,规范才是

核心假设：

当生成代码的成本趋近于零,代码只是规范的编译产物(Compiled Artifact)。

Martin Fowler 的三级模型：

L1 规范先行 Spec-First

先写规范,再让 AI 生成代码

L2 规范锚定 Spec-Anchored

规范与代码同步维护,规范是"锚"

L3 规范即源码 Spec-as-Source

人类只编辑规范,永远不直接改代码

既然审查 AI 生成的代码太累,不如在源头把规矩定死。造砖免费的时代,最值钱的是蓝图。

跃迁 3 — 巨头下注与工具落地

在新世界里重建秩序

三家巨头几乎同时推出落地工具——这在 AI 编程领域极为罕见：

Tessl

Guy Podjarny / Snyk 创始人

10,000+ 开源库规范注册表,解决 API 幻觉

Amazon Kiro

内建 SDD 工作流的代理 IDE

三阶段开发 + 代理钩子

GitHub Spec Kit

官方规范驱动工具

三步流程 + 不可变原则

ThoughtWorks 技术雷达将 SDD 列为值得关注的趋势——但也警告：规范文件同样冗长难审查,各工具诠释差异大,标准尚未统一。

跃迁 3 — 项目指令文件：SDD 的自发大规模采纳

千万开发者已经在做 SDD 而不自知

开发者自发在项目目录建 markdown 文件,告诉 AI "在这个项目里应该怎么工作"。各平台只是把这个草根行为收拢为正式功能——Anthropic 叫 CLAUDE.md,OpenAI 叫 AGENTS.md,Cursor 叫 Rules。名字不同,本质一样：**在 AI 写代码之前,先给它明确的约束和上下文。**这就是最轻量的 SDD。



当前主流层级：多数团队 L1,L2 开始普及

跃迁 3 — 过渡地带：我们正在这里

规范管意图,谁来管执行?



SDD 解决了

- AI 应该写什么代码(意图约束)
- 规范作为团队共享的"锚"
- 减少了"黑箱代码"的问题



SDD 还没解决

- 多个代理怎么分工协作(编排)
- 代理出错了怎么自动纠正(反馈回路)
- 代理需要什么工具和权限(环境基础设施)

规范告诉代理**要做什么**,但没有告诉代理**怎么高效地做**。给工人一张蓝图但没有给他工厂——他知道要造什么,但缺乏工具、流水线和质检体系。

三家工具诠释各异、标准未统一,根因是底层模型还在快速进化——今天为 **GPT-4 级代理**设计的规范框架,面对 **GPT-5 级代理**可能需要重写。

跃迁 4 — 代理工程 Agentic Engineering (2026.2)

Karpathy 亲手"杀死"了 Vibe Coding

"我现在最喜欢的名字是'代理工程'——'代理'是因为 99% 的时间你不再直接写代码,而是编排代理; '工程'是强调这里面有艺术、科学和专业性。"

Karpathy 的判断: "LLM 代理能力在 2025 年 12 月跨过了连贯性阈值(Threshold of Coherence),引发了软件工程的相变。"



代理自主操作次数: **20 次**(半年前 10 次,翻倍)

创业公司 vs 企业采纳: **33% vs 13%**

跃迁 4 — 发散的前沿：三大学派并存

第四次跃迁尚未定局



纪律派

代表：Osmani(Google)

核心主张：架构先行、编排代理、无情测试

人类角色：工程经理



治具派

代表：OpenAI(Harness Engineering)

核心主张：建造让代理高效工作的基础设施

人类角色：工厂建造者



激进派

代表：Boris Cherny(Anthropic)

核心主张：代理自建工具、代码是临时的

人类角色：方向守望者

前三次跃迁已经“盖棺定论”。第四次仍在“百家争鸣”——三个学派各执己见,正是因为下一代模型可能让某一派立刻过时。

你的团队该站哪一派？取决于你的项目本质、团队认知水平、和你能承受多大的变化速度。没有标准答案。

跃迁 4 — 纪律派：Addy Osmani (Google)

AI 生成的代码永远是草稿，不是成品

Osmani 提出从**指挥者 (Conductor)** 到 **编排者 (Orchestrator)** 的演进路径，被 O'Reilly 收录为行业参考框架。Conductor 阶段人与单个代理紧密配合，实时反馈，逐步迭代；Orchestrator 阶段人协调多个代理并行工作，从微观管理转向宏观调度。



纪律派四条铁律：

规范先于代码 (Specs before Code)	小块迭代 (Small Chunks)	代码即草稿 (Code as Draft)	你署名你负责 (Accountability)
"15 分钟的瀑布"胜过 2 小时的返工	给代理聚焦的小任务，而非整个代码库	永远不要未经审查就合并 AI 生成的代码	你提交的代码出了安全和可维护性问题，不能怪 AI

纪律派的核心信念：**模型在变强，但人类的判断力不能退化。** 问题不再是用不用，而是有没有纪律地用。

跃迁 4 — 治具派：OpenAI 的震撼实验

不要修代理,要修环境

📌 3 人 + GPT-5 → 5 个月 → 100 万行代码(0 人类手写)→ 节省 10 倍时间

四大治具组件：

上下文工程	架构约束 Linter	AGENTS.md	GC Agents
代理改数据库时自动抓最新 Schema——90% 的失败是喂的信息不对	"所有 API 必须过鉴权"——违反自动驳回	分层指令,代理开工前自动读取	后台 AI 定期巡检僵尸代码、文档漂移

治具派的核心信念：**成功的 AI 代码不靠无限自由度,而靠约束解空间(Constraining the Solution Space)**。治具是对抗模型更迭的最佳防御——不和基础模型对赌,但必须建立自己的特制流水线。

跃迁 4 — 激进派：Boris Cherny (Anthropic)

不要和大模型对赌

Boris Cherny 代表了最激进的立场：完全拥抱模型进化的速度，不对抗变化，而是让系统本身具备适应能力。



Boris 个人 AI 生成比例

IDE 已卸载

Anthropic 人均产出变化

代理拓扑 (Agent Topologies)

Boris Cherny 的团队使用了一种独特的代理拓扑：给 Claude 一个 Spec + Asana 看板，Claude 便能自建 ticket，spawn sub-agents，并让 agents 自行认领任务。这种方法甚至能让 agent swarm 在一个周末内跑出 Plugins 功能，展现了惊人的效率和自适应能力。

Boris 的三条激进判断

脚手架就是技术债 你今天搭的脚手架，六个月后就是技术债。	指令要极简 模型变强后需要的指引越来越少。	Agent 学会自建工具 工厂在学会改造自己。
---------------------------------	--------------------------	----------------------------

激进派的核心信念：不要和大模型对赌。 代码保质期只有几个月，唯一的防御是拥抱变化本身。

代理时代的新现实（一）：10倍差距

从人力成本到算力成本的拐点

维度	过去	代理时代
核心成本	高薪开发者(线性扩编)	API Token + AgentOps 工具(算力杠杆)
人均产出	1-2 个 PR/天	10-20 个 PR/天
组织形态	庞大单一职能团队	极少数架构精英 + 多面手通才

先行者与后来者的差距已从百分比扩大到数量级。

代理时代的新现实（二）：三大未解难题

新世界的规则还没写完

代理工程带来了生产力的跃升,但同时也打开了潘多拉魔盒。三个核心问题至今没有行业共识,每个组织都在摸索自己的答案。



安全 (Security)

代理越强大,攻击面越大。自主代理可以调用 API、修改数据库、部署代码——一旦被攻破或被误导,后果难以预测。



治理 (Governance)

谁为代理生成的代码负责? 当 AI 写了 95% 的代码,出了安全漏洞或合规问题,责任如何界定? 法律和组织流程都还没跟上。



人才培养 (Talent Development)

当编码技巧不再是核心竞争力,新一代工程师怎么培养? 如果不写代码就能“做软件”,那计算机教育该教什么?

这些问题没有标准答案。每个组织必须根据自己的风险承受能力、行业特性和文化,找到属于自己的平衡点。

代理时代的新现实（三）：控制权的重构

在更高的抽象层重建秩序



正题(2024 觉醒)

看到 AI 的极大潜力



反题(2025 狂热)

完全放弃控制,被现实毒打



合题(2026 编排)

用规范约束意图,用治具约束行为——在更高的抽象层重建控制权

但这个"合题"本身还没有定论。螺旋还在上升,终局未定。

结语：唯一确定的事

Karpathy 说："代理工程里面有艺术、科学和专业性。"

Boris Cherny 说："最重要的能力是新手心态——承认自己错了,然后重新出发。"

正如这些洞见所揭示，今天我们讨论的所有“答案”——无论是 SDD、治具工程，还是代理拓扑——都只是当下之解，它们自身也有保质期。

世上没有一劳永逸的框架可供盲目套用。您的团队真正需要的，并非固定的正确答案，而是在持续变化中磨砺并建立自身判断力的能力。

在这个技术迭代如飞、代码保质期仅为数月的时代，唯一不会过期的，是您拥抱新知的速度，以及在不确定性中做出果断判断的勇气。